Find Missing Number in a Sequence

```
class MissingNumberFinder():
    @staticmethod
    def find_single_missing(arr):
        n = len(arr) + 1
        expected_sum = n * (n + 1) // 2
        actual_sum = sum(arr)
        return expected_sum - actual_sum

    @staticmethod
    def find_multiple_missing(arr, n):
        full_set = set(range(1, n + 1))
        arr_set = set(arr) # for fast membership checks and set difference operations.
        return list(full_set - arr_set)


numbers = [1, 3, 4, 5, 6]
print("Single missing:", MissingNumberFinder.find_single_missing(numbers))

numbers2 = [1,2,4,6]
print("Multiple Missing:", MissingNumberFinder.find_multiple_missing(numbers2, 6))
```

Output:

Single missing: 2

Multiple Missing: [3, 5]

- **Data Structures**
  - **List (**arr**):** Used to store the sequence of numbers.
  - **Set (**full_set **and** arr_set**):** Used in find_multiple_missing for fast membership checks and set difference operations.
- **Algorithms**

1. find_single_missing
   - Uses the **mathematical formula** for the sum of the first $n$natural numbers:

$$\text{Sum}(1..n) = \frac{n(n+1)}{2}$$

- Subtracts the actual sum of the array from the expected sum to find the missing number.
- **Algorithm type**: Arithmetic + aggregation.

2. find_multiple_missing
   - Builds a set of all numbers from 1 to n.
   - Converts the array into a set.

    - Finds the difference between the two sets to identify missing numbers.
    - **Algorithm type**: Set operations (difference).

- **Time Complexity**
  - find_single_missing
    - sum(arr) → $O(n)$
    - Formula computation → $O(1)$
    - **Overall:** $O(n)$
  - find_multiple_missing
    - Constructing full_set → $O(n)$
    - Converting arr to arr_set → $O(n)$
    - Set difference → $O(n)$
    - **Overall:** $O(n)$
- **Space Complexity**
  - find_single_missing
    - Only a few integer variables.
    - **Overall:** $O(1)$
  - find_multiple_missing
    - Stores two sets of size up to n.
    - **Overall:** $O(n)$